

ArcSDM user manual

Spatial Data Modeler 6 for ArcGIS Pro 3.0+

GTK ArcSDM development team

2026-06-24

Table of contents

Introduction	3
What is ArcSDM 6?	3
History of ArcSDM	3
Installation	4
Tools within the ArcSDM toolbox	5
Exploratory Data Analysis	7
Area Frequency Table	7
Principal Component Analysis (Raster)	8
Principal Component Analysis (Vector)	9
Predictive Modelling	10
Multi-Layer Perceptron	10
Predict Classifier	10
Predict Regressor	11
Test MLP Classifier	11
Test MLP Regressor	12
Train MLP Classifier	13
Train MLP Regressor	16
Weights of Evidence	17
Agterberg-Cheng CI Test	19
Calculate Response	19
Calculate Weights	20
Log WofE Details	22
Preprocessing	22
Raster Processing	22
Reclass & Fuzzify	22
Vector Processing	23
Split Points	23
Thin Points	23
Validation	24
Calculate ROC Curves and AUC Values	24
References	26

Introduction

What is ArcSDM 6?

ArcSDM 6 is an ArcGIS Pro based Spatial Data Modelling toolbox with focus on mineral prospectivity modelling. It contains tools for data preprocessing, exploratory data analysis and predictive modelling. Although the toolbox is designed with mineral prospectivity modelling in mind, it can be used for other types of spatial predictive modelling as well.

History of ArcSDM

The Spatial Data Modeler (SDM) toolbox was originally developed as a joint Geological Survey of Canada (GSC) – U.S. Geological Survey (USGS) project supported by mining companies and geological surveys. The original Arc-SDM was written in Avenue for ESRI ArcView and Spatial Analyst. It was developed by Laura Kemp at the GSC under the direction of Graeme Bonham-Carter (GSC) and Gary Raines (USGS) (Kemp et al., 2001). The toolbox provided tools for Weights of Evidence, logistic regression, fuzzy logic.

In 2004, Arc-SDM 2 was released for ArcGIS 8.3, developed by Don Sawatzky under the direction of Gary Raines (USGS) and Graeme Bonham-Carter (GSC) (Sawatzky et al., 2004). The code was rewritten from Avenue to Visual Basic, as ArcGIS no longer supported Avenue. Arc-SDM 3 followed for ArcGIS 9.0, also developed by Don Sawatzky under the direction of Raines and Bonham-Carter. ArcSDM 3.1 was released for ArcGIS 9.1 as a beta version with the same toolset.

Since 2005, the code was maintained by Prof. Carlos Roberto de Souza Filho at the University of Campinas (UNICAMP), Brazil. Some of the previous ArcSDM versions and associated documentation are still available from the UNICAMP website (<https://www.ige.unicamp.br/sdm/>). ESRI Inc. took over the maintenance of the Fuzzy Logic tools, which are now included in the Spatial Analyst toolbox in ArcGIS.

In 2016, the Geological Survey of Finland (GTK) began recoding the toolbox as version ArcSDM 5 for ArcGIS 10.4 in Python. The current version, ArcSDM 6, targets ArcGIS Pro 3.0 and later. It introduces new tools such as Principal Component Analysis and Multilayer Perceptron (MLP) neural networks, including support for Synthetic Minority Oversampling Technique (SMOTE) to address class imbalance in MLP training. The latest stage of development of ArcSDM 6 has included modification of legacy code to meet the current standards, improved version control through git and replacement of ArcGIS Geoprocessing with ArcPy. The ArcSDM toolbox is currently available through GTK's GitHub page (<https://github.com/gtkfi/ArcSDM>).

Experimental tools toolbox:

ArcSDM 6 contains also separate experimental tools toolbox that has been developed by the GTK. The toolbox contains tools for mineral prospectivity modeling and data processing that are still at an experimental stage of development. A separate user manual will be published for the experimental tools toolbox in future.

Installation

The most up-to-date installation instructions can be found on the ArcSDM GitHub page at <https://github.com/gtkfi/ArcSDM>. As ArcSDM is an ArcGIS Pro toolbox, ArcGIS Pro version 3.0 or newer should be installed on the system. Thus, system requirements are the same as for ArcGIS Pro version 3.0 or newer (see ArcGIS Pro documentation for system requirements).

Tools within the ArcSDM toolbox

ArcSDM toolbox covers common tools for mineral prospectivity modelling. Tools are hierarchically organized within the toolbox, major tool groups being: [Exploratory Data Analysis](#), [Predictive modelling](#), [Preprocessing](#) and [Validation](#). Although, typical workflow would often include preprocessing and exploratory data analysis steps before the predictive modelling and model validation, these tools will be described in this manual in same alphabetical order as they occur within the ArcSDM toolbox. The structure of the toolbox is illustrated in tree chart below:

ArcSDM Toolbox structure

```

ArcSDM/
├── Exploratory Data Analysis/
│   ├── Area Frequency Table
│   ├── Principal Component Analysis (Raster)
│   └── Principal Component Analysis (Vector)
├── Predictive Modelling/
│   ├── Multilayer Perceptron/
│   │   ├── Predict Classifier
│   │   ├── Predict Regressor
│   │   ├── Test MLP Classifier
│   │   ├── Test MLP Regressor
│   │   ├── Train MLP Classifier
│   │   └── Train MLP Regressor
│   └── Weights of Evidence/
│       ├── Agterberg-Cheng CI Test
│       ├── Calculate Response
│       ├── Calculate Weights
│       └── Log WoE Details
├── Preprocessing/
│   ├── Raster Processing/
│   │   └── Reclass & Fuzzify/
│   │       └── Reclass & Fuzzify
│   └── Vector Processing/
│       ├── Split Points
│       └── Thin Points
└── Validation/
    └── Calculate ROC Curves and AUC Values
  
```

Throughout the manual, inline code is color-coded to help distinguish different types of items at a glance: **Tool Name** — tool names are highlighted with a red background. **Input parameter** — tool input parameters are highlighted with a blue background. **Output name** — outputs like elements of printed messages, saved files, and fields within output tables are highlighted with a green background.

Each tool within the ArcSDM toolbox takes input parameters from the user. These parameters can define input rasters, the output file location, or they can affect how the tool is executed. For example, Figure 1 shows the input parameters for the **Principal Component Analysis (Raster)** tool. The tool has fields (parameters) for input raster layers, one field for the output file

location (**Transformed Data**) and few fields that control how the data is processed before Principal Component Analysis (PCA) and how many principal components are chosen.

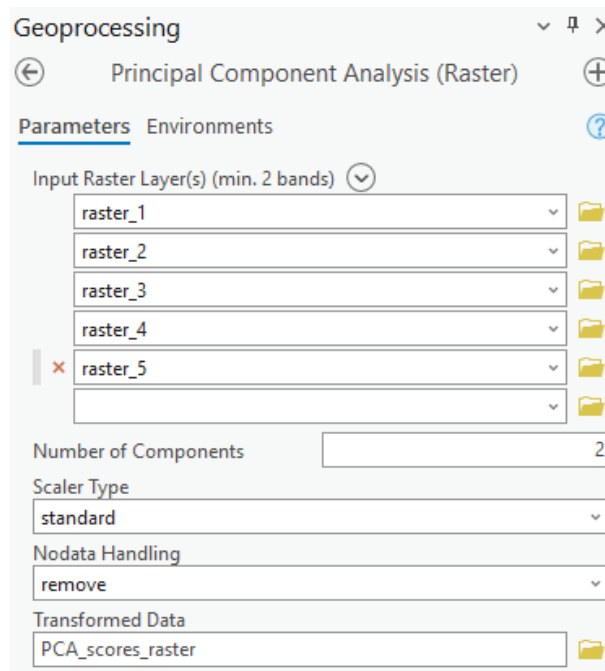


Figure 1: Inputs for the Principal Component Analysis (Raster) tool.

In this manual, the input parameters for the each tool are presented in blue boxes (see example below).

Input parameters of the tool

- **Input Parameter/Field**: Description about the input parameter/field.

Tools will also produce output messages that can be viewed in ArcGIS Pro by clicking View Details at the bottom of the Geoprocessing panel (Figure 2).

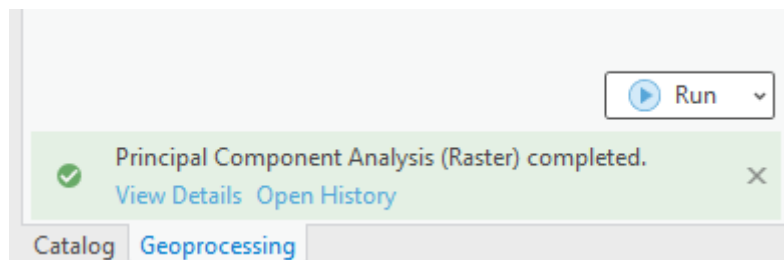


Figure 2: Viewing output message of the Principal Component Analysis (Raster) tool.

In this manual, examples of output messages produced by some tools are shown in green boxes (see example below).

Messages produced by the tool

Message produced by the tool.

Exploratory Data Analysis

Exploratory Data Analysis (EDA) in ArcSDM provides tools that help users understand the structure, distribution, and relationships within their spatial data before predictive modeling. The main EDA tools available are:

- **Area Frequency Table**: Summarizes the spatial association between evidence rasters and known training or validation sites (e.g., mineral occurrences), allowing users to assess how well each evidence layer discriminates between areas with and without deposits. This also helps in evaluating the performance of a predictive model by allowing construction of so called Prediction-Area (P-A) plot. This plot evaluates model effectiveness by plotting the prediction rate (the percentage of known occurrences/deposits correctly identified) against the occupied area (the percentage of total map deemed prospective).
- **Principal Component Analysis (Raster)** and **Principal Component Analysis (Vector)**: Principal Component Analysis (PCA) transforms a high-dimensional dataset into a smaller set of orthogonal (uncorrelated) variables called principal components. These principal components are ordered so that the first component captures the largest possible amount of variance in the data, the second captures the next largest amount (subject to being orthogonal to the first), and so on. With only a few components, PCA often retains most of the important information from the original dataset, providing an effective low-dimensional representation. PCA is available for both raster and vector data, both with similar functionality but different input types.

Area Frequency Table

Area Frequency Table creates a table that summarizes the spatial relationship between an evidence raster and training or validation site locations. For each class in the evidence raster, the tool calculates the area (in km²) and the number of sites that fall within that class. It then computes cumulative area and cumulative site percentages, which can be used to assess how well the evidence raster discriminates between areas with and without deposits. The tool also calculates an efficiency metric based on the area under the cumulative curve.

Input parameters of the Area Frequency Table

- **Training sites**: Location of training points as vector layer.
- **Input Raster Layer**: Evidence raster to evaluate.
- **Value field** (*default value*: **VALUE**): Attribute field of the raster used as class identifier.
- **Unit area (km2)**: Area of the unit cell in km².
- **Output table**: Filepath where the output table is saved.

The output table contains the following fields:

- **RASTERVALU**: The raster class value.
- **Area_sqkm**: Area of the class in square kilometers.
- **Frequency**: Number of training sites within the class.
- **CAPP_CumAr**: Cumulative area percentage (ascending order).
- **Eff_CumAre**: Cumulative area percentage (descending order, used for efficiency).
- **Cum_Sites**: Cumulative site percentage (descending order).
- **I_CumSites**: Inverse cumulative site percentage (100 - Cum_Sites).
- **Eff_AUC**: Area under the cumulative curve for each class interval.

Principal Component Analysis (Raster)

Principal Component Analysis (Raster) conducts PCA with raster input. It outputs PCA scores for each component as multiband raster, with one band representing each component. Printed message gives information about the PCA, like loadings and explained variances.

Input parameters of the Principal Component Analysis (Raster)

- **Input Raster Layers(s)**: Input features as raster layers. If only one raster layer is used, it must have at least two bands. If single band rasters are used, at least two input rasters are needed.
- **Number of Components**: Number of principal components that are kept.
- **Scaler Type** (*default value: standard*): Type of scaler which will be used. Can be either **standard**, **min_max**, or **robust**.
 - **standard**: Features are centered by subtracting the mean and scaled to have unit variance. This transformation results in features with a mean of 0 and a standard deviation of 1.
 - **min_max**: Features are scaled linearly based on their minimum and maximum values so that they fall within a range of [0,1].
 - **robust**: Features are centered by subtracting the median and scaled according to the interquartile range ($IQR = Q3 - Q1$). Because scaling uses median and IQR, this method is robust to outliers.
- **Nodata Handling** (*default value: remove*): How missing data values are treated. Can be either **remove** or **replace**. **remove** drops all cells/rows that contain missing data before performing PCA. **replace** replaces missing data values with the column mean before performing PCA.
- **Transformed Data** (*default value: PCA_scores_raster*): Output filepath. Default is to save as **PCA_scores_raster** layer in project filegeodatabase.

Principal Component Analysis (Raster) produces messages that give information about the PCA results. The **Variance explained** table lists each principal component together with its eigenvalue, the percentage of variance it explains, and the cumulative percentage explained by all components up to that point. The **Loadings table** lists the input variables, i.e., raster layers or raster bands, together with their loading values for each retained principal component. Large positive or negative loading values indicate that the variable has a strong influence on that

component, while values close to zero indicate a weaker contribution. The variables are ordered by the principal component on which they have the strongest absolute loading, which helps highlight the dominant contribution of each variable. Example message produced by **Principal Component Analysis (Raster)** below:

Messages produced by the Principal Component Analysis (Raster)

```
===== PCA results =====
Transformed data is saved in C:\\Users\\...\\project.gdb\\PCA_scores_raster

Variance explained:
PC          Eigenvalue  Variance %  Cumulative %
-----
PC1          1.6130       32.3        32.3
PC2          1.3013       26.0        58.3
PC3          0.9683       19.4        77.7
-----

Loadings table:
Variable          PC1          PC2          PC3
-----
FaultsDens        +0.8097     -0.3176     -0.0492
FaultsDist        -0.7475     +0.4441     -0.1358
AM_RTP_AnomDist   -0.5236     -0.6069     -0.0327
AR_AnomDist       -0.2888     -0.5107     +0.7263
RG_TDR            +0.2025     +0.6116     +0.6471
-----
```

Principal Component Analysis (Vector)

Principal Component Analysis (Vector) conducts similar PCA as **Principal Component Analysis (Raster)**, however it differs by taking vector input instead raster.

Input parameters of the Principal Component Analysis (Vector)

- **Input Vector**: Input vector layer containing the features for PCA.
- **Select Fields**: Fields of the vector layer that are used for the PCA.
- **NoData Value** (*default value: -99*): Integer value that is considered to represent NoData.
- **Number of Components**, **Scaler Type**, **Nodata Handling**: Are same as in **Principal Component Analysis (Raster)**. See [Principal Component Analysis \(Raster\)](#) for descriptions.
- **Transformed Data** (*default value: `PCA_scores_vector`*): Output filepath. Default is to save as `PCA_scores_vector` layer in project filegeodatabase.

Principal Component Analysis (Vector) produces identical printout to **Principal Component Analysis (Raster)**. See [Principal Component Analysis \(Raster\)](#) for further details about the message.

Predictive Modelling

The Predictive Modelling section contains tools for creating mineral prospectivity models. Two approaches are implemented: Multi-Layer Perceptron (MLP), a machine learning method capable of capturing non-linear relationships and interactions between evidence layers (i.e., input rasters), and Weights of Evidence (WofE), a probabilistic Bayesian method that combines evidence layers additively under a conditional independence assumption.

Multi-Layer Perceptron

Multi-Layer perceptron (MLP) is a feedforward network consisting on layers of fully connected neurons. MLP has input layer, output layer and one or more hidden layers between these two. Neurons in the hidden layers use non-linear activation functions, which gives MLP the ability to learn non-linear function approximations. MLP has previously been utilized as a method for mineral prospectivity modelling, see e.g., (Singer and Kouda, 1996; Brown et al., 2000). ArcSDM provides six MLP tools covering training, testing, and prediction for both classification and regression tasks.

Predict Classifier

Predict Classifier uses trained MLP model for prediction i.e., to generate prospectivity maps. The tool outputs two rasters, binary classification raster and probability raster. The classification raster has either a value of 1 or 0 for the every cell of the raster. Value 1 indicates that the cell is considered prospective and value 0 indicates that cell is not considered prospective. The probability raster contains decimal values between 0 and 1 for each cell. These values are the MLP's predicted probabilities and indicate how likely it is that a given cell belongs to class 1, i.e., the cell is considered prospective. Note that classification raster is derived from the probability raster by thresholding with the user defined threshold (default value is 0.5).

Input parameters of the Predict Classifier

- **Input features**: Input features as raster layers.
- **Input Feature NoData Value** (*default value: -99*): No data value to indicate cells with no data.
- **Input Model File**: File that contains the trained MLP model. By default trained models are saved into same folder as project file geodatabase with the name `model.keras`.
- **Classification threshold** (*default value: 0,5*): Value between 0 and 1 which is used for thresholding predicted probabilities to produce binary output raster.
- **Output predicted values probability raster**: Raster that has predicted probability values for each cell, indicating how likely cell belongs to class 1.
- **Output predicted values classified raster**: Binary raster that is derived from the **Output predicted values probability raster** by thresholding with the user defined **Classification threshold**.

Predict Regressor

Predict Regressor is otherwise identical to **Predict Classifier**, with couple differences: the **Classification threshold** parameter is absent, and loss options together with validation metrics are different. Instead of a probability raster and a classified raster, it outputs one continuous-valued raster (**Output predicted values raster**) where each cell contains the regression prediction.

Test MLP Classifier

Test MLP Classifier evaluates the performance of the trained MLP Classifier based on a test dataset supplied by the user. Test dataset is defined with the fields **Input Features**, **Target Labels**, and **Target Labels attribute**. User is able to select which of the four **Test Metrics** will be computed for the model. Each of the four test metrics is expressed in terms of true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN). Here, positive and negative refer to class labels (class 1 and class 0, respectively), applied to both the model's predictions and the ground truth. True and false indicate whether the predicted label matched the ground truth. Hence, true positives (TP) are cases where the model correctly predicted the positive class, false positives (FP) are cases where the model incorrectly predicted the positive class, true negatives (TN) are cases where the model correctly predicted the negative class, and false negatives (FN) are cases where the model incorrectly predicted the negative class.

Accuracy indicates the proportion of predictions that were correct, whether positive or negative. Accuracy is defined as:

$$\text{Accuracy} = \frac{\text{correct classifications}}{\text{total classifications}} = \frac{TP + TN}{TP + TN + FP + FN}$$

Recall measures the proportion of actual positive cases that the model correctly identifies. Recall is defined as:

$$\text{Recall} = \frac{\text{correctly classified positives}}{\text{all actual positives}} = \frac{TP}{TP + FN}$$

Precision reflects how reliable the model is when it predicts positive class. Precision is defined as:

$$\text{Precision} = \frac{\text{correctly predicted positives}}{\text{all predicted positives}} = \frac{TP}{TP + FP}$$

Both recall and precision in isolation cannot give reliable indication about model performance. For example, perfect recall of 1 could be achieved by classifying every cell positive. However, precision would then be really poor because many predicted positives would be incorrect. To address this, performance metric known as F1 score can be calculated by taking the harmonic mean of precision and recall. It balances precision and recall into single number between 0 and 1 where higher value indicates better performance. F1 score is defined as:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Input parameters of the Test MLP Classifier

- **Input features**, **Input Feature NoData Value**, **Input Model File**, **Classification threshold**, **Output predicted values probability raster**, **Output predicted values classified raster**: Are same as in Predict Classifier. See [Predict Classifier](#) for descriptions.
- **Target Labels**: Target labels of the test data.
- **Target Labels Attribute**: Attribute that defines the target label.
- **Label NoData Value**: Value that indicates no data in the target labels.
- **Test Metrics**: Different test metrics that can be computed for the model. Can be either **accuracy**, **precision**, **recall** or **f1**.

Test MLP Classifier produces printed message shows model performance on user selected test metrics.

Test MLP Classifier

```
Starting MLP classifier test...
Classifier test completed.
Metrics: {'accuracy': 0.77778, 'precision': 0.68132, 'recall': 0.89855, 'f1': 0.775}
```

Test MLP Regressor

Test MLP Regressor evaluates the performance of the trained MLP Regressor based on a test dataset supplied by the user. Test dataset is defined with the fields **Input Features**, **Target Labels**, and **Target Labels attribute**. Evaluating regression models involves measuring how well a model's predictions match the actual continuous values it aims to estimate. ArcSDM provides four different **Test Metrics** metrics to evaluate regression models.

Mean Squared Error (MSE) is the average of the squared differences between predicted and actual values. MSE is defined as:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Root Mean Square Error (RMSE) is the square root of MSE. Taking the square root returns the error to the original units of the target variable, making the metric easier to interpret. RMSE is defined as:

$$\text{RMSE} = \sqrt{\text{MSE}}$$

Mean Absolute Error (MAE) represents the average of absolute differences between predictions and actual values. MAE is defined as:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Coefficient of Determination (R^2) measures how well the model explains the variability of the target variable. R^2 of 1 means that model explains all of the variability while R^2 of 0 means that model explains none of the variability. R^2 is defined as:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Within the formulas the predicted value is $\hat{y}$, ground truth value is represented with y and mean of the ground truth with $\bar{y}$.

Save output raster to a file is the filepath where the output raster with continuous-valued predicted values is saved. **Test Metrics** can be one of **mse**, **rmse**, **mae** or **r2**, as described above. Test MLP Regressor produces printed message shows model performance on user selected test metrics similarly to **Test MLP Classifier**.

Train MLP Classifier

Train MLP Classifier trains the MLP model for binary classification and saves it into disk for further use. User can define the training strategy and MLP architecture. Some of the choices available for the user are explained in more detail below:

With the **Neurons Per Layer** field it is possible to define number of hidden layers and number of neurons within each hidden layer. Thus, input is given in comma separated integers where number of integers equals to number of hidden layers and integer values itself define how many neurons are to be used within each hidden layer. Three different activation functions (**relu**, **sigmoid** and **tanh**) are available for hidden layer activation function. The Rectified Linear Unit (**relu**) activation function passes positive values and outputs zero for negative values. Both **sigmoid** and **tanh** (hyperbolic tangent) are S-shaped activation functions that squash the inputted values within limited ranges. In the case of **sigmoid** the output range is [0,1] and in the case of **tanh** the output range is [-1, 1]. These activation functions are defined as follows:

$$\text{ReLU}(x) = \max(0, x)$$

$$\text{Sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

$$\text{tanh}(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Epochs, **Batch Size**, **Optimizer**, **Learning rate**, **Early Stopping** and **Early Stopping Patience** define the training strategy. **Epochs** defines the number of epochs that are used during training. Each epoch corresponds to one complete pass through the entire training dataset. **Batch Size** determines how many samples are processed before the model updates its parameters. Instead of using the entire dataset at once, the data is split into smaller batches, which are used for training. Larger batch sizes tend to lead to faster training (per epoch) and smoother gradients, while smaller batch sizes lead to slower training and noisier gradients. **Optimizer** specifies the algorithm used to update the weights based on the computed gradients. Four optimizers are available:

- **sgd**: Stochastic Gradient Descent, updates weights using the gradient of the loss with respect to each mini-batch.
- **adam**: Adaptive Moment Estimation, combines momentum and per-parameter adaptive learning rates.
- **adagrad**: Adapts learning rate per parameter based on accumulated past gradients.
- **rmsprop**: Uses exponentially decaying average of squared gradients to normalize the learning rate per parameter.

Learning Rate controls how large the update steps are when adjusting the model's weights. A small learning rate results in slower but potentially more stable convergence, while a large learning rate can speed up training but may cause instability or overshooting.

Early Stopping is a regularization technique used to prevent overfitting. It monitors the model's performance on a validation dataset during training and stops the process when the performance (validation loss) stops improving. **Early Stopping Patience** defines how many epochs the training will continue without improvement before stopping. This allows the model some time to recover from temporary plateaus before training is terminated.

Some binary classification problems have often unbalanced dataset where, one class is important but rare like mineral deposit locations in prospectivity modelling. In such cases it can be beneficial to address class imbalance by generating more synthetic minority class. This helps machine learning models to learn better decision boundaries and reduce bias toward the majority class (e.g., non deposit locations). One approach to generate more synthetic samples is the Synthetic Minority Oversampling Technique (SMOTE). It generates new synthetic examples of the minority class, rather than simply duplicating existing ones. It works by selecting, for each minority class sample, one of its k nearest minority class neighbors at random, and then generating a synthetic minority class sample by interpolating between the two points in feature space. The interpolation is also random, as the new sample is placed at a randomly chosen position along the line connecting the two points. **Train MLP Classifier** has an option to utilize SMOTE by activating **Apply SMOTE** checkbox, which opens fields for SMOTE options. User can define **Number of Synthetic Samples** to be generated, **Minority Class Label** and **Number of Nearest Neighbors (k)** that are used in synthetic sample generation.

Input parameters of the Train MLP Classifier

- **Input features**: Input features of the training data as raster layers.
- **Target Labels**: Target labels of the training data. Point locations of presence (positive) and absence (negative) instances.

- **Target Labels Attribute**: Attribute that defines the target label.
- **Input Feature NoData Value**: Indicates which value in the input features is considered as NoData.
- **Label NoData Value**: Indicates which value in the target labels is considered as NoData.
- **Neurons per layer**: A comma separated list of values that define number of neurons within each hidden layer. MLP network will have as many hidden layers as there is values in the list.
- **Validation split** (*default value: 0,2*): Decimal number between 0 and 1, which determines the proportion of training data that will be used for model validation. Ignored if **Validation Data** is provided.
- **Validation Data**: Separate validation dataset for model validation, point locations of presence (positive) and absence (negative) instances. If provided, overrides **Validation split**.
- **Activation function** (*default value: relu*): Determines the activation function used in hidden layers. Can be either **relu**, **sigmoid** or **tanh**.
- **Epochs** (*default value: 50*): Number of epochs.
- **Batch Size** (*default value: 32*): Batch size used for training.
- **Optimizer** (*default value: adam*): Optimizer used for training. Options are **adam**, **adagrad**, **rmsprop** or **sgd**.
- **Learning rate** (*default value: 0,001*): Initial learning rate used during training.
- **Dropout rate** (*default value: 0,0*): Proportion of neurons to be randomly dropped (set to zero) during each training step. Dropout is a regularization technique that helps prevent overfitting by forcing the network to not rely on any single neuron.
- **Early Stopping** (*default value: checked*): Checkbox which controls whether early stopping is used or not.
- **Early Stopping Patience** (*default value: 5*): Number of allowed consecutive training epochs where validation loss doesn't improve.
- **Validation Metrics** (*default value: accuracy*): Can be either **accuracy**, **precision**, or **recall**. These are explained in [Test MLP Classifier section](#).
- **Random State**: Sets the seed for the random number generation. Using same random state gives reproducible results.
- **Apply SMOTE**: Checkbox which controls whether synthetic samples are generated with SMOTE.
- **Number of Synthetic Samples**: How many new synthetic minority class samples are generated with SMOTE.
- **Minority Class Label** (*default value: 1*): Label of the minority class that is used for synthetic sample generation.
- **Number of Nearest Neighbors (k)** (*default value: 5*): Specifies how many nearest neighbors are considered when generating synthetic samples.
- **Output Model File** (*default value: model*): Filepath where the trained model is saved.

Train MLP Classifier produces messages about the training progress. Example message below, where the model was trained for just two epochs to keep the message concise. The message contains information about the progress of the training, providing for each epoch the current progress percentage, training loss and accuracy, as well as validation loss and validation accuracy. These values describe how the model's performance evolves during training and how well it

generalizes to unseen data.

Train MLP Classifier

```
Starting MLP classifier training...
Adding hidden layers...
Splitting data...
Training the model...
Training started.
Epoch 1:
Progress: 50.00%
    loss: 0.7114818096160889
    accuracy: 0.0
    val_loss: 0.7091069221496582
    val_accuracy: 0.0
Epoch 2:
Progress: 100.00%
    loss: 0.7039183974266052
    accuracy: 0.0
    val_loss: 0.7016670107841492
    val_accuracy: 0.20000000298023224
Training completed.
===== Model training completed. =====
Saved model to: C:\Users\...\model.keras
Training history: {'loss': [0.7114818096160889, 0.7039183974266052],
'accuracy': [0.0, 0.0], 'val_loss': [0.7091069221496582, 0.7016670107841492],
'val_accuracy': [0.0, 0.20000000298023224]}
```

Train MLP Regressor

Train MLP Regressor trains the MLP regression model and saves it into disk for further use. It differs from **Train MLP Classifier** by having additional **Last Layer Activation Function** and **Loss Function** parameters, as well as different validation metrics. As it is otherwise similar to **Train MLP Classifier**, only the differing parameters are described here.

Last Layer Activation Function defines the activation function used in the output layer of the regressor. Options are **linear** (default) or **sigmoid**. For regression tasks, **linear** is typically appropriate as it allows the network to output any continuous value.

Three different **Loss Functions** are available for the MLP Regression model. These are **mse**, **mae** and **huber** losses. For the descriptions of **mse** and **mae**, see the **Test MLP Regressor** section for description. The huber loss is a robust loss function that blends the properties of mse and mae. For small prediction errors, huber behaves like mse with a quadratic penalty. For large errors, it transitions to a linear penalty like mae, making it less sensitive to outliers than mse. The point where the loss switches from **mse** like to **mae** like behavior is controlled by the threshold parameter delta, which in ArcSDM is fixed to 1.0. **huber** loss is defined as follows:

$$\text{huber} = \begin{cases} \frac{1}{2} (y_i - \hat{y}_i)^2, & \text{if } |y_i - \hat{y}_i| \leq \delta, \\ \delta (|y_i - \hat{y}_i| - \frac{1}{2}\delta), & \text{if } |y_i - \hat{y}_i| > \delta. \end{cases}$$

Validation Metrics for the regressor are regression-specific and can be one of `mse`, `mae` or `rmse`, as described in the [Test MLP Regressor](#) section.

Weights of Evidence

Weights of evidence (WofE) is a data-driven, Bayesian statistical method commonly used in mineral prospectivity modelling (Bonham-Carter, 1989; Bonham-Carter et al., 1989; Agterberg et al., 1990, 1993; Raines, 1999; Nykänen et al., 2008). It measures the spatial association between evidence maps and point data like mineral deposit locations. WofE is based on a log-linear form of Bayes' rule and assumes that evidence maps are conditionally independent with respect to deposits. Internally, the method treats each evidence layer as a binary pattern (presence or absence of a feature). The input evidence rasters can be multiclass (e.g., lithological units or binned distance classes), and the tool converts them into binary patterns during the weight calculation. WofE is conducted based on the evidence rasters and mineral deposit location data (points) as follows:

1. The study area is divided into unit cells of a user-defined size. Each deposit is assumed to occupy exactly one unit cell, as long as unit cell size is sufficiently small. To enforce this assumption, training points should be thinned so that no two points fall within the same unit cell (see [Thin Points](#)). The prior probability is estimated as the ratio of deposit-bearing unit cells D to the total number of unit cells T . Prior probability is used as a starting point for every cell of the prediction, ie. prospectivity map. Prior probability is defined as follows:

$$P(D) = \frac{D}{T}.$$

For convenience, probability units are converted to logits:

$$L(D) = \ln\left(\frac{D}{T - D}\right)$$

2. Weights for the evidence rasters are calculated based on the ratios of conditional probabilities. They are based on presence B or absence $\bar{B}$ of the pattern within binary evidence rasters (1 or 0, respectively) and presence or absence of the deposit. Both positive W^+ and negative W^- weights are calculated for every evidence raster:

$$W^+ = \ln\left(\frac{P(B | D)}{P(B | \bar{D})}\right)$$

$$W^- = \ln\left(\frac{P(\bar{B} | D)}{P(\bar{B} | \bar{D})}\right)$$

In plain terms, the conditional probabilities can be interpreted as follows:

$$P(B | D) = \frac{\text{deposit-bearing unit cells where evidence is present}}{\text{total deposit-bearing unit cells}}$$

$$P(B | \bar{D}) = \frac{\text{deposit-free unit cells where evidence is present}}{\text{total deposit-free unit cells}}$$

$$P(\bar{B} | D) = \frac{\text{deposit-bearing unit cells where evidence is absent}}{\text{total deposit-bearing unit cells}}$$

$$P(\bar{B} | \bar{D}) = \frac{\text{deposit-free unit cells where evidence is absent}}{\text{total deposit-free unit cells}}$$

3. The calculated weights are assigned to every cell. The final prospectivity map, i.e. prior logits $L(D)$ are updated by adding information from the evidential rasters to yield posterior logit. This update is possible by simple addition because the weights, being log-likelihood ratios, are already expressed in logit space:

$$L(D | B) = L(D) + \sum_{i=1}^n W_i$$

Where each W_i is W_i^+ if the evidence is present and W_i^- if the evidence is absent.

4. As a final step, posterior logits can be converted to posterior probability values for map display:

$$P(D | B) = \frac{e^{L(D|B)}}{1 + e^{L(D|B)}}$$

In summary, the WofE method assigns a prior logit to every cell, and then for each binary evidence raster either W_i^+ or W_i^- is added to the cell depending on whether the evidence is present or absent. The final posterior logits are then converted into posterior probability values.

Practical considerations when using WofE include the following:

WofE tools within ArcSDM require user to provide a mask that defines the study area where the WofE calculations are conducted. The mask can be set within geoprocessing environment settings, both vector and raster layers can be used as mask. The area covered by the mask is used in calculations while empty areas within evidence rasters are filled with **Missing data value** and ignored in calculations. Therefore, it doesn't matter even if there are slight differences in the extent of the evidence rasters. All WofE tools should be used with same **Unit area (km2)**, **Training sites** and **Missing data value** parameters to ensure that the calculations are consistent across different tools.

Agterberg-Cheng CI Test

WofE modelling can be sensitive to violations of conditional independence of evidence maps. If evidence are not independent, the posterior probabilities will be inflated as the same evidence is counted more than one time. Strongly correlated evidence maps can be e.g., different derivatives that are computed from the same source evidence map or geochemical evidence maps based on elements that have similar characteristics. **Agterberg-Cheng CI Test** measures the conditional independence of evidence maps and with the test it is possible to evaluate if conditional independence is violated (Agterberg and Cheng, 2002). In practical terms, the test compares the sum of posterior probabilities (S) with the observed number of deposits (D). The standard deviation of S is then used to assess whether the difference ($S - D$) is statistically significant.

Input parameters of the Agterberg-Cheng CI Test

- **Post Probability raster**: Posterior probability raster produced by the **Calculate Response** tool (the **W_pprb** output).
- **Probability Std raster**: Standard deviation raster produced by the **Calculate Response** tool (the **W_std** output).
- **Training sites**: Location of training points as vector layer.
- **Unit area (km²)**: Area of the unit cell in km². Should be the same value that was used in **Calculate Response** and **Calculate Weights**.
- **Output CI Test File**: Filepath where the conditional independence test results are saved as a text file (optional).

Calculate Response

Calculate Response produces the prospectivity map and associated rasters which give information about uncertainty. **Calculate Response** tool outputs following rasters:

- **Output post probability raster**: Posterior probabilities, i.e., prospectivity map.
- **Output standard deviation raster**: Standard deviation of weights (uncertainty).
- **Output MD variance raster**: Variance due to the missing data.
- **Output Total Std Deviation Raster**: The combined standard deviation of weights and missing data.
- **Output confidence raster**: Studentized posterior probability.

Input parameters of the Calculate Response

- **Input Raster Layer(s)**: Evidence map as raster layer
- **Input weights tables**: Weight table(s) computed earlier with **Calculate Weights** tool. There should be one weight table for each input raster layer.
- **Training sites**: Location of training points as vector layer
- **Ignore missing data**: If checked, cells with the missing data value are excluded from the output rasters. If unchecked, missing data is propagated through the calculations.
- **Missing data value** (default value: **-99**): Value that identifies the missing data.
- **Unit area (km²)** (default value: **1**): Area of the unit cell
- **Output post probability raster** (default value: **W_pprb**): Filepath for the probability

- ```
raster
```
- **Output standard deviation raster** (default value: **W\_std**): Filepath for the standard deviation raster
  - **Output MD variance raster** (default value: **W\_MDvar**): Filepath for the MD variance raster
  - **Output Total Std Deviation raster** (default value: **W\_Tstd**): Filepath for the total standard deviation raster
  - **Output confidence raster** (default value: **W\_conf**): Filepath for the confidence raster

Message printed by the **Calculate Response** tool includes lots of information of inputs and different stages of calculation. As printed message is very long, it is not shown here.

## Calculate Weights

**Calculate Weights** calculates weights and associated statistics for single evidence raster. Evidence raster must be single band raster with different unique classes. **Type** parameter controls how weights are calculated for the evidence raster. If evidence raster classes represent different nominal categories (e.g., lithologies) that have not ordering, then **Categorical** should be chosen for the **Type**. This will treat each category as independent binary raster to calculate weights and associated statistics for each category. If categories represent some continuous value (e.g., distance to fault or element concentration) that has been binned into different classes, statistics can be calculated cumulatively. Options **Ascending** and **Descending** do this in respective orders. **Ascending** should be used when classes corresponding to small values e.g., distance to the fault are considered most prospective. **Descending** should be used if the opposite is true, i.e., classes corresponding to the large values e.g., elemental concentration are the most prospective.

**Unit area (km2)** Defines the size of the unit cell area used in calculations. The output of WofE is a map showing the probability that the unit cell area contains a deposit. The calculated weight values are relatively independent of the chosen unit cell area, as long as it is relatively small. However, probability values will change as it affects especially to the prior probability.

The output of **Calculate Weights** is saved into File Geodatabase Table and it contains following statistics for each class in a evidence raster:

- **AREA\_SQ\_KM**: Class area in square kilometers.
- **AREA\_UNITS**: Class area in chosen unit area.
- **NO\_POINTS**: Number of points within a class.
- **W+**: Positive weight of a class.
- **W+ Std**: Standard deviation of the positive weight.
- **W-**: Negative weight of a class.
- **W- Std**: Standard deviation of the negative weight.
- **Contrast\_**: The difference between W+ and W-.
- **Contrast\_Std**: The standard deviation of contrast.
- **Studentized\_Contrast**: The studentized contrast: the contrast divided by its standard deviation.
- **Generalized\_Class**: Class value 2 for prospective and 1 for everything else.
- **Generalized\_Weight**: Generalized W+ weight.

- **Generalized\_Weight\_Std**: Standard deviation for generalized weight.

From these statistics, standard deviations of weights measure the uncertainty associated in weights. Contrast ( $C$ ) is a overall measure of spatial association of a evidence raster and deposits. Contrast measures how well the pattern predicts the training sites. It can be calculated by taking the difference between weights:

$$C = W^+ - W^-$$

A positive  $W^+$  indicates that the presence of evidence favours deposits, while a negative  $W^-$  indicates that the absence of evidence disfavours deposits. A large positive contrast indicates a strong positive association between the evidence and deposits.

Studentized contrast is a measure of significance of contrast. It is a ratio of the contrast ( $C$ ) to the standard deviation of contrast  $s(C)$ .

**Generalized\_Class**, **Generalized\_Weight** and **Generalized\_Weight\_Std** are produced when **Type** is **Ascending** or **Descending**. They represent the optimal binarization of the cumulative classes: the class with the highest significant contrast defines the split point, dividing the evidence raster into a prospective group (class 2) and a non-prospective group (class 1).

#### Input parameters of the Calculate Weights

- **Evidence raster layer**: Evidence raster
- **Evidence raster codefield**: Optional text field from the evidence raster attribute table to use as class labels instead of the default VALUE field.
- **Training points feature**: Location of training points as vector layer.
- **Type**: How weights are calculated. Can be either **Descending**, **Ascending** or **Categorical**.
- **Output weights table**: Filepath to save calculated weights and associated statistics.
- **Confidence Level of Studentized Contrast** (*default value: 2*): Threshold for studentized contrast used in **Ascending** and **Descending** modes. Classes with studentized contrast above this value are considered spatially associated with deposits, while classes below are grouped together.
- **Unit area (km2)** (*default value: 1*): Area of the unit cell.
- **Missing data value** (*default value: -99*): Value that indicates missing data.

**Calculate Weights** prints out many useful details related to weight calculation. Details include for example mask area, number of unit cells within the area and prior probability. Full printout below:

#### Messages produced by the Calculate Weights

```
Starting weight calculation

Setting overwriteOutput to True

===== ArcSDM version & environment info =====
 ArcSDM 6.0 for ArcGIS Pro
```

```

Arcgis environment: ArcGISPro (3.5.6)
Workspace: C:\arc_projects\wofe\wofe.gdb (LocalDatabase)
Scratch workspace: C:\arc_projects\wofe\wofe.gdb (LocalDatabase)
Mask area (km^2): 22263.54314406652

===== WofE parameters =====
Mask size (km^2): 22263.54314406652
Unit Cell Size: 0.04
Training Sites: 51
Unit Cell Area: 0.04 km^2, Cells in area: 556588.578601663
Prior Probability: 0.000092
Training Points: C:\arc_projects\wofe\wofe.gdb\deposits
Study Area Raster: C:\arc_projects\wofe\wofe.gdb\Study_area_extent
Study Area Area: 22263.54314406652 km^2
===== Calculate weights =====
Creating table: C:\arc_projects\wofe\wofe.gdb\evidence_raster1_CA (Ascending)
Created output weights table

```

## Log WofE Details

**Log WofE Details** provides printout of WofE details similarly to **Calculate Weights** tool. Thus, it contains details about study area size and prior probability etc. As the printout is similar to **Calculate Weights** it is not shown here. The printout is saved as txt file into filepath given by user.

### Input parameters of the Log WofE Details

- **Training sites**: Location of the training points as vector layer.
- **Unit area (km2)** (*default value: 1*): Area of the unit cell.
- **Log results to a file**: Filepath where the txt file is saved.

## Preprocessing

### Raster Processing

#### Reclass & Fuzzify

With the **Reclass & Fuzzify Tool** it is possible to convert numerical rasters into fuzzy membership values. Inputted categorical evidence raster can be either categorical or numerical. In case of continuous numerical raster, it can be first categorized i.e., binned.

### Input parameters of the Reclass & Fuzzify Tool

- **Categorical evidence raster**: Categorical evidence raster to be classified.
- **Reclass field**: Field where reclassification is based.
- **Reclassification**: Remap table that defines the mapping from old values to new values. This is populated automatically based on the selected evidence raster and reclass field.

- **FM Categorical** Filepath where output raster with fuzzy membership values is saved.
- **Divisor** New class value is divided with the divisor to get the fuzzy membership values.

## Vector Processing

### Split Points

**Splitting Tool** Divides the sample site points into separate training and testing portions. The size of training dataset is controlled with the **Training Fraction** parameter. User given percentage of points are randomly selected for training points while rest of the points are assigned into test points.

#### Input parameters of the Splitting Tool

- **Input Points**: Location of the input points as vector layer.
- **Training Fraction**: Proportion of samples randomly assigned into training dataset.
- **Output training layer**: Output location of training points as vector layer.
- **Output testing layer**: Output location of testing points as vector layer.

Printout of the **Splitting Tool** gives information about the number of points assigned to training and testing points.

#### Messages produced by the Splitting Tool

```
Starting Splitting Tool

Total features: 51
Training features: 30
Testing features: 21
Creating output layer: reduced_sites_test
Output layer created: reduced_sites_test
Splitting completed successfully.
```

### Thin Points

**Thinning Tool** selects a spatially thinned subset of points such that no two retained points are closer than a specified minimum distance. The minimum distance can be provided directly or derived from a user-defined unit area. When calculated from unit area, the distance is computed assuming each point represents a circular area, using  $d = \sqrt{A/\pi}$ , where  $A$  is the area per point. If both **Unit area** and **Minimum distance (meters)** are specified, the minimum distance value is used.

#### Input parameters of the Thinning Tool

- **Input Points**: Location of the input points as vector layer.
- **Unit area**: Size of the unit cell.

- **Area unit**: Area unit for the unit area.
- **Minimum distance (Meters)**: What is the minimum distance of the closest points in meters.
- **Output layer**: Filepath for the thinned points output.

Printout message gives information about the used minimum distance and how many points were retained from input points after thinning.

### Messages produced by the Thinning Tool

```
Starting Thinning Tool

Input layer linear unit: Meter
Calculated minimum distance (meters): 564.19
Minimum distance in layer units: 564.19 Meter
Selected 49 thinned points out of 51 input points.
Creating output layer: thinned_sites
Output layer created: thinned_sites
```

## Validation

### Calculate ROC Curves and AUC Values

**Calculate ROC Curves and AUC Values** tool evaluates the quality of prediction rasters by computing Receiver Operating Characteristic (ROC) curves and Area Under the Curve (AUC) scores. ROC curves describe the relationship between the True Positive Rate (*TPR*) and the False Positive Rate (*FPR*) over a range of threshold values applied to the prediction raster. *TPR* and *FPR* are defined as:

$$TPR = \text{Recall} = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$

where *TP* is the number of true positives, *FN* is the number of false negatives, *FP* is the number of false positives, and *TN* is the number of true negatives.

By varying the threshold, the tool assesses how well the model can distinguish between positive and negative classes across different decision boundaries.

The AUC score summarizes the area under the ROC curve as a single value between 0 and 1. Higher AUC values indicate better model performance, meaning the model is more capable of correctly ranking positive instances higher than negative ones. A value close to 0.5 suggests performance comparable to random guessing. Together, ROC curves and AUC scores provide a threshold-independent measure of model performance.

The **Calculate ROC Curves and AUC Values** tool evaluates the predictive performance of spatial models using presence-absence data. It takes as input **Presence Locations**, which represent



points where the phenomenon of interest (for example, mineral deposits) is known to occur, and optionally [Absence Locations](#), which represent points where the phenomenon is known to be absent. In addition, the tool requires one or more raster datasets that represent model predictions, such as prospectivity maps. When both presence and absence locations are provided, the tool samples the raster values at these locations and directly computes the Receiver Operating Characteristic (ROC) curve and the Area Under the Curve (AUC) for each model. These metrics describe how well the model distinguishes between presence and absence locations across different classification thresholds.

If absence locations are not provided, the tool estimates ROC curves and AUC values using a Monte Carlo simulation approach. In this case, pseudo-absence locations are generated randomly within the raster extent, with the number of pseudo-absence points equal to the number of presence points. This random sampling process is repeated 10,000 times, and for each iteration a ROC curve and corresponding AUC value are calculated. The final ROC curve is constructed by aggregating the results of all simulations. A fixed grid of False Positive Rate (FPR) values ranging from 0 to 1 is defined, with a resolution equal to ten times the number of presence points. For each FPR value, the median True Positive Rate (TPR) across all simulations is computed, producing a central ROC curve that represents the typical model performance. In addition, 95% confidence intervals are derived for both the ROC curve and the AUC values based on the distribution of simulation results.

The tool outputs a ROC plot comparing all input models, along with AUC values and, when pseudo-absences are used, their associated confidence intervals [Figure 3](#). It also creates a results table containing ROC coordinates and summary statistics, enabling detailed comparison of model performance even when true absence data is unavailable.

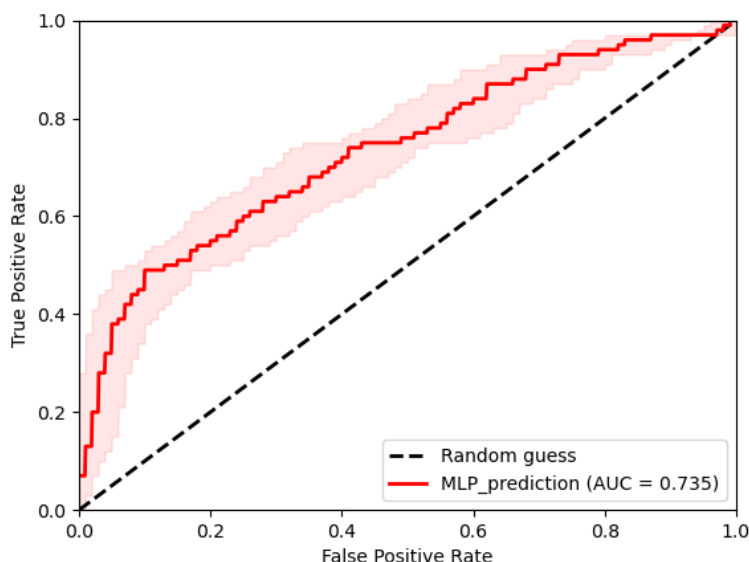


Figure 3: ROC Curve with AUC score.

### Input parameters of the Calculate ROC Curves and AUC Values Tool

- **Presence locations**: Point locations where the phenomenon of interest (e.g., mineral deposits) is known to occur.
- **Absence locations**: Absence point locations where the phenomenon is known not to occur (optional). If not provided, pseudo-absences are generated using Monte Carlo simulation.
- **Prediction raster(s)**: One or more prediction rasters (e.g., prospectivity maps) to evaluate.
- **Destination Folder**: Folder where the ROC curve plot and results table are saved.

Example printout shows how ROC and AUC were calculated only by providing presence locations and pseudo-absences were generated randomly.

### Messages produced by the Calculate ROC Curves and AUC Values

```
Performing Monte Carlo simulation (10000 realizations).
Realization #1000 (AUC = 0.710)
Realization #2000 (AUC = 0.714)
Realization #3000 (AUC = 0.725)
Realization #4000 (AUC = 0.779)
Realization #5000 (AUC = 0.719)
Realization #6000 (AUC = 0.682)
Realization #7000 (AUC = 0.745)
Realization #8000 (AUC = 0.741)
Realization #9000 (AUC = 0.756)
Realization #10000 (AUC = 0.735)
Completed Monte Carlo simulation.
MLP_prediction: AUC = 0.735.
MLP_prediction: 95% confidence interval = 0.691-0.775.
Saved ROC curve plot to 'C:\Users\...\results.png'.
Saved results database table to 'C:\...\results.dbf'.
```

## References

- Agterberg, F., Bonham-Carter, G., Cheng, Q., and Wright, D., 1993, Weights of evidence modeling and weighted logistic regression for mineral potential mapping: *Computers in geology*, v. 25, p. 13–32.
- Agterberg, F., Bonham-Carter, G.F., and Wright, D., 1990, Statistical pattern integration for mineral exploration, *in* *Computer applications in resource estimation*, Elsevier, p. 1–21.
- Agterberg, F.P., and Cheng, Q., 2002, Conditional independence test for weights-of-evidence modeling: *Natural resources research*, v. 11, p. 249–255.
- Bonham-Carter, G.F., 1989, Weights of evidence modelling: A new approach to mapping mineral potential: *Statistical applications in the earth sciences*, p. 171–183.

- Bonham-Carter, G.F., Agterberg, F.P., and Wright, D.F., 1989, Integration of geological datasets for gold exploration in nova scotia: Digital geologic and geographic information systems, v. 10, p. 15–23.
- Brown, W.M., Gedeon, T., Groves, D., and Barnes, R., 2000, Artificial neural networks: A new method for mineral prospectivity mapping: Australian journal of earth sciences, v. 47, p. 757–770.
- Kemp, L.D., Bonham-Carter, G.F., Raines, G.L., and Looney, C.G., 2001, Arc-SDM: ArcView extension for spatial data modelling using weights of evidence, logistic regression, fuzzy logic and neural network analysis: Geological Survey of Canada / U.S. Geological Survey, <http://www.ige.unicamp.br/sdm/>.
- Nykänen, V., Groves, D., Ojala, V., and Gardoll, S., 2008, Combined conceptual/empirical prospectivity mapping for orogenic gold in the northern fennoscandian shield, finland: Australian Journal of Earth Sciences, v. 55, p. 39–59.
- Raines, G.L., 1999, Evaluation of weights of evidence to predict epithermal-gold deposits in the great basin of the western united states: Natural Resources Research, v. 8, p. 257–276.
- Sawatzky, D.L., Raines, G.L., Bonham-Carter, G.F., and Looney, C.G., 2004, ArcSDM2: ArcMAP extension for spatial data modelling using weights of evidence, logistic regression, fuzzy logic and neural network analysis; <http://www.ige.unicamp.br/sdm/ArcSDM2/>.
- Singer, D.A., and Kouda, R., 1996, Application of a feedforward neural network in the search for kuroko deposits in the hokuroku district, japan: Mathematical Geology, v. 28, p. 1017–1023.